Artifact Appendix

A. Artifact Overview

This artifact provides a reproducible local benchmark pipeline for multi-hop social graph queries in a cloud-native setting.
The primary evaluation scope contains two report-ready benchmark tracks:

1. `facebook_full` on `Neo4j + PostgreSQL`
2. `twitter_top10000` on `Neo4j + PostgreSQL`

`JanusGraph` is included only as a secondary validation/enhancement path and is not part of the primary comparison matrix.

Public repository: https://github.com/bigyong-996/cs5296-graph-multihop-optimization

B. Artifact Inventory

Core scripts:

- Data preparation: `scripts/prepare_dataset.py`
- Backend loading: `scripts/load_neo4j.py`, `scripts/load_postgres.py`, `scripts/load_janusgraph.py`
- Benchmark execution: `scripts/run_benchmark.py`
- Post-processing: `scripts/aggregate_results.py`, `scripts/plot_results.py`

Supporting documentation:

- Reproduction runbook: `docs/runbook.md`
- Result interpretation and claim boundaries: `docs/experiment_notes.md`
- Demo-oriented execution narrative: `docs/demo_script.md`

C. Environment and Dependencies

Required environment:

- OS: macOS or Linux
- Python: `3.12+`
- Container runtime: Docker + Docker Compose

Services used:

- Primary: `Neo4j`, `PostgreSQL` (containerized)
- Optional: `JanusGraph + Cassandra` profile

Setup:

```
python3 -m venv .venv
source .venv/bin/activate
python3 -m pip install -e ".[dev]"
docker compose -f infra/docker-compose.yml up -d neo4j postgres
```

D. Reproduction Procedure

D.1 Functional verification (minimal checks)

```
python3 -m pytest -q
python3 scripts/smoke_test_core_backends.py
python3 scripts/smoke_test_core_backends.py --node-id 964 --left-id 0
        --right-id 1
python3 scripts/smoke_test_core_backends.py --node-id 21409847 --left-
        id 12 --right-id 13
```

D.2 Primary benchmark track 1 (`facebook_full`)

```
python3 scripts/prepare_dataset.py --input
        datasets/raw/facebook_combined.txt --dataset-name
        facebook_full --symmetrize --output-dir
        datasets/derived/facebook_full
python3 scripts/load_neo4j.py --dataset-dir
        datasets/derived/facebook_full
python3 scripts/load_postgres.py --dataset-dir
        datasets/derived/facebook_full
python3 scripts/run_benchmark.py --backend neo4j --dataset-name
        facebook_full --workload-file benchmarks/facebook_full.json --
        output results/raw/neo4j-facebook-full.jsonl
python3 scripts/run_benchmark.py --backend postgres --dataset-name
        facebook_full --workload-file benchmarks/facebook_full.json --
        output results/raw/postgres-facebook-full.jsonl
python3 scripts/aggregate_results.py --input results/raw/neo4j-
        facebook-full.jsonl --output results/summary/neo4j-facebook-
        full.csv
```

```
python3 scripts/aggregate_results.py --input results/raw/postgres-
        facebook-full.jsonl --output results/summary/postgres-
        facebook-full.csv
python3 scripts/plot_results.py --input results/summary/facebook-full-
        comparison.csv --output results/summary/facebook-full-p50-
        latency.png
```

D.3 Primary benchmark track 2 (`twitter_top10000`)

```
curl -L https://snap.stanford.edu/data/twitter_combined.txt.gz -o
        datasets/raw/twitter_combined.txt.gz
gzip -dc datasets/raw/twitter_combined.txt.gz >
        datasets/raw/twitter_combined.txt
python3 scripts/prepare_dataset.py --input
        datasets/raw/twitter_combined.txt --dataset-name twitter --
        symmetrize --max-nodes 10000 --output-dir
        datasets/derived/twitter_top10000
python3 scripts/load_neo4j.py --dataset-dir
        datasets/derived/twitter_top10000
python3 scripts/load_postgres.py --dataset-dir
        datasets/derived/twitter_top10000
python3 scripts/run_benchmark.py --backend neo4j --dataset-name
        twitter_top10000 --workload-file
        benchmarks/twitter_top10000.json --output results/raw/neo4j-
        twitter-top10000.jsonl
python3 scripts/run_benchmark.py --backend postgres --dataset-name
        twitter_top10000 --workload-file
        benchmarks/twitter_top10000.json --output
        results/raw/postgres-twitter-top10000.jsonl
python3 scripts/aggregate_results.py --input results/raw/neo4j-
        twitter-top10000.jsonl --output results/summary/neo4j-twitter-
        top10000.csv
python3 scripts/aggregate_results.py --input results/raw/postgres-
        twitter-top10000.jsonl --output results/summary/postgres-
        twitter-top10000.csv
python3 scripts/plot_results.py --input results/summary/twitter-
        top10000-comparison.csv --output results/summary/twitter-
        top10000-p50-latency.png
```

D.4 Optional JanusGraph validation (secondary)

```
docker compose -f infra/docker-compose.yml --profile janusgraph up -d
        cassandra janusgraph
python3 scripts/load_janusgraph.py --dataset-dir
        datasets/derived/facebook_tiny
python3 scripts/smoke_test_janusgraph.py --node-id 1
```

E. Expected Outputs

Primary evidence bundle:

- `results/summary/facebook-full-comparison.csv`

- `results/summary/facebook-full-p50-latency.png`
- `results/summary/twitter-top10000-comparison.csv`
- `results/summary/twitter-top10000-p50-latency.png`

Supporting outputs:

- Raw benchmark traces in `results/raw/*.jsonl`
- Backend-specific summary tables in `results/summary/`

F. Verification Criteria

Reproduction is considered successful if:

1. Tests and smoke checks complete without runtime errors.
2. Both primary benchmark tracks generate raw JSONL outputs.
3. Summary CSV and p50 plots are produced for both datasets.
4. Observed trends match the report-safe claims in `docs/experiment_notes.md`.

G. Claim Boundaries and Caveats

The main comparison is `Neo4j` online traversal versus `PostgreSQL + closure_3` (bounded precomputation).
This is a valid systems-level comparison, but not a strict engine-only apples-to-apples benchmark.
`JanusGraph` should be presented as a secondary validated path rather than part of the primary two-backend matrix.

H. Practical Reproducibility Notes

- Each backend container keeps one active dataset at a time; reload datasets before switching tracks.
- If full reruns are time-constrained, the committed `results/summary/` outputs can be used as reference evidence.

I. Two Reproduction Methods

To make evaluation easier, this artifact supports two complementary reproduction methods.

Method A: Script-First Docker Reproduction (Primary)

- This is the canonical path in Sections C–D.
- It uses CLI scripts directly (`prepare_dataset`, `load_*`, `run_benchmark`, `aggregate_results`, `plot_results`).
- It is recommended for strict end-to-end reproducibility checks of the full pipeline.

Method B: Notebook-First Local Reproduction (Demo-Oriented)

- Notebook entrypoint: `outputs/cs5296-demo/CS5296_local_oneclick_demo.ipynb`
- This path wraps the same pipeline into one linear notebook flow:

1. Preflight checks (Python, Docker, Compose, daemon)
2. Service startup (`Neo4j + PostgreSQL`)
3. Tiny live run (`facebook_tiny`) for real-time demonstration
4. Live artifact generation (JSONL -> CSV -> p50 plot)
5. Report-scale evidence display (`facebook_full`, `twitter_top10000`)

Method B is intended for classroom/demo recording convenience, while Method A remains the primary script-level reproduction reference for formal artifact verification.